Comunicación por paso de mensajes en SD

Respuestas teóricas y análisis del caso práctico · Proyecto **OrderFlow** · SO – Sistemas Distribuidos

Preguntas teóricas

1. ¿Qué ventajas tiene el paso de mensajes frente a la memoria compartida en sistemas distribuidos?

En un sistema distribuido los procesos corren en máquinas distintas que no comparten memoria física, así que el paso de mensajes es el modelo natural: refleja lo que realmente ocurre (datos viajando por la red) en vez de simular una memoria común.

- **Desacoplamiento:** cada proceso tiene su propio estado privado y solo expone una interfaz de mensajes. Se puede cambiar la implementación interna, el lenguaje o la base de datos de un servicio sin afectar a los demás.
- **Sin sincronización de memoria:** no hacen falta locks, semáforos ni protocolos de coherencia de caché distribuida, que son caros y propensos a errores (condiciones de carrera, deadlocks).
- **Escalabilidad:** agregar nodos no exige mantener una vista coherente de una memoria global; basta con que los nuevos nodos sepan enviar y recibir mensajes.
- **Aislamiento de fallos:** si un proceso falla, no corrompe el estado de los demás. El fallo se manifiesta como un mensaje que no llega o una respuesta que no vuelve, y se puede manejar con timeouts y reintentos.
- **Heterogeneidad:** nodos con distinto hardware, sistema operativo o lenguaje se entienden si acuerdan el formato del mensaje (JSON, Protobuf, etc.).
- **Comunicación explícita:** los puntos de interacción son visibles, lo que facilita el monitoreo, el registro y la depuración.

2. Diferencias entre comunicación síncrona y asíncrona por mensajes

	Síncrona	**Asíncrona**
Comportamiento del emisor	Envía y queda bloqueado hasta recibir la respuesta (request/response).	Envía y sigue trabajando; la respuesta, si la hay, llega después o nunca.
Disponibilidad requerida	Emisor y receptor deben estar activos al mismo tiempo.	El receptor puede estar caído; un intermediario guarda el mensaje.
Acoplamiento	Temporal y espacial alto: el emisor conoce la dirección del receptor y depende de su tiempo de respuesta.	Bajo: el emisor solo conoce el broker o la cola, no quién consume ni cuándo.
Latencia percibida	Se suma la de toda la cadena de llamadas.	El emisor responde rápido; el trabajo se completa más tarde.
Manejo de fallos	El error se propaga en cascada hacia el que llamó.	Se absorbe con reintentos, colas y dead-letter queues.
Consistencia	Inmediata: el emisor sabe el resultado al instante.	Eventual: los demás servicios se enteran con cierto retraso.
Ejemplos	REST, gRPC unario	RabbitMQ, Kafka, colas en general

La síncrona conviene cuando el emisor *necesita* la respuesta para decidir (en OrderFlow, saber si hay stock antes de confirmar). La asíncrona conviene para avisar que algo ocurrió sin esperar a nadie (enviar el correo, actualizar métricas).

3. ¿Qué problemas pueden aparecer en la comunicación por mensajes?

Problema	Qué ocurre	Mitigaciones habituales
Pérdida	El mensaje no llega por una caída de red, del broker o del receptor antes de procesarlo.	Colas y mensajes persistentes, confirmaciones del broker (publisher confirms), ack del consumidor solo después de procesar, reintentos.
Duplicación	Un reintento o una reentrega hace que el mismo mensaje se procese dos veces (por ejemplo, el consumidor procesó pero se cayó antes del ack).	Consumidores idempotentes: identificador único por mensaje y registro de los ya procesados.
Desorden	Los mensajes llegan en un orden distinto al de envío (varias rutas, varias particiones, reintentos).	Particionar por clave para que los mensajes relacionados vayan al mismo canal ordenado; números de secuencia o timestamps.
Latencia	Demoras variables de red o de procesamiento; en llamadas síncronas encadenadas se acumulan.	Timeouts, comunicación asíncrona donde no se necesita respuesta, caché.
Fallos parciales	No se sabe si el otro extremo procesó el mensaje o no (timeout ambiguo).	Operaciones idempotentes y reintentos seguros, circuit breakers.
Mensajes venenosos	Un mensaje que siempre falla bloquea la cola si se reencola indefinidamente.	Límite de reintentos y dead-letter queue.
Sobrecarga	El productor genera más rápido de lo que el consumidor procesa.	Colas como buffer, prefetch/backpressure, más consumidores en paralelo.
Incompatibilidad de formato	Un servicio cambia el esquema del mensaje y rompe a los consumidores.	Contratos versionados, esquemas (Protobuf, Avro), cambios compatibles hacia atrás.

4. Diferencia entre un Message Queue (RabbitMQ) y un Event Streaming Platform (Kafka)

	Message Queue (RabbitMQ)	Event Streaming (Kafka)
Modelo	Cola de tareas: cada mensaje se entrega a **un** consumidor y se borra al confirmarse.	Log distribuido de solo agregado: los eventos quedan guardados y cada consumidor lee a su ritmo.
Retención	Hasta que se consume (ack).	Por tiempo o tamaño configurable, independientemente de quién lo leyó.
Quién lleva la cuenta	El broker sabe qué mensajes faltan entregar.	Cada consumer group guarda su offset (posición en el log).
Varios consumidores	Hay que crear una cola por interesado y enrutarla con exchanges.	Cada consumer group recibe todos los eventos sin configurar nada en el productor.
Relectura	No: un mensaje consumido ya no existe.	Sí: se puede volver a cualquier offset y reprocesar la historia.
Orden	Por cola (se pierde con varios consumidores en paralelo).	Garantizado dentro de cada partición.
Enrutamiento	Muy flexible (direct, topic, fanout, headers), prioridades, TTL, DLQ nativas.	Simple: topic y partición por clave.
Fortaleza	Distribuir trabajo, baja latencia por mensaje, lógica de entrega rica.	Alto volumen, múltiples lectores, auditoría, event sourcing, analítica.

En síntesis: RabbitMQ piensa en **mensajes como tareas a entregar**; Kafka piensa en **eventos como hechos a registrar**.

5. ¿Qué rol cumple un middleware de mensajería?

Es la capa intermedia que se ubica entre productores y consumidores y se encarga de transportar los mensajes para que las aplicaciones no tengan que resolverlo por su cuenta. Sus funciones principales son:

- **Desacoplar** en espacio (el productor no conoce la dirección del consumidor) y en tiempo (no necesitan estar activos a la vez).
- **Almacenar y garantizar la entrega:** persistencia, confirmaciones, reintentos y colas de mensajes fallidos.
- **Enrutar** los mensajes hacia los destinatarios correctos (exchanges, topics, suscripciones).
- **Amortiguar picos de carga:** la cola actúa como buffer cuando el consumidor es más lento que el productor.
- **Escalar:** permite agregar consumidores en paralelo para repartir el trabajo.
- **Ofrecer observabilidad:** métricas de colas, mensajes pendientes, consumidores conectados.

Gracias al middleware, cada servicio se concentra en su lógica de negocio y la complejidad de la comunicación confiable queda en una pieza especializada.

Ejercicio 1 – Comparación gRPC frente a REST

En OrderFlow, `OrdersService` consulta por REST a `InventoryService` al confirmar un pedido: primero `GET /products/:id/availability` y luego `POST /products/:id/reservations`. La reserva vuelve a validar el stock, porque entre la consulta y la reserva otro pedido podría haberlo consumido.

> **Mediciones en el prototipo.** Confirmación normal: **19 ms**. Con 800 ms de demora artificial en InventoryService: **1,63 s**, porque las dos llamadas encadenadas suman la demora dos veces. Con InventoryService caído o con 2,5 s de demora: **503 a los 2,0 s** (timeout configurado) y el pedido sigue en `CREATED`. Esto muestra el acoplamiento temporal de la comunicación síncrona: la disponibilidad y la latencia de OrdersService dependen de las de InventoryService.

Aspecto	REST (implementado)	gRPC
Transporte y formato	HTTP/1.1, JSON en texto.	HTTP/2, Protocol Buffers en binario: mensajes más chicos y serialización más rápida.
Contrato	Implícito o documentado aparte (OpenAPI opcional). Un campo mal escrito se detecta en tiempo de ejecución.	Obligatorio en un archivo `.proto`; se genera el código cliente y servidor con tipos, así que los errores de contrato aparecen al compilar.
Rendimiento	Suficiente para este caso de bajo volumen.	Menor latencia y uso de CPU; HTTP/2 multiplexa muchas llamadas sobre una conexión. Se nota con miles de consultas por segundo.
Modelo de llamada	Recursos y verbos HTTP (`GET`, `POST`).	Llamada a procedimiento remoto: `CheckAvailability(productId, quantity)`. Encaja mejor con una consulta entre servicios internos.
Streaming	No nativo.	Unario, streaming del servidor, del cliente y bidireccional (por ejemplo, avisos de cambios de stock en vivo).
Timeouts	Hay que implementarlos a mano (en el prototipo con `AbortSignal.timeout`).	Deadlines integrados que se propagan a lo largo de la cadena de llamadas.
Depuración y herramientas	Muy simple: curl, navegador, Postman. El mensaje se lee a simple vista.	Requiere herramientas específicas (grpcurl, Postman con soporte gRPC); el binario no es legible.
Compatibilidad con navegadores	Total.	Necesita gRPC-Web y un proxy.

Conclusión para este escenario. La consulta de stock es una comunicación interna entre servicios, frecuente, con un contrato estable y que está en el camino crítico de la confirmación. gRPC sería la opción técnicamente más adecuada en producción: menor latencia por llamada, contrato tipado compartido y deadlines propagados. REST se eligió para el prototipo por su simplicidad, porque se prueba con curl y porque a este volumen la diferencia de rendimiento no es significativa. Hay que destacar que **ninguno de los dos elimina el acoplamiento temporal**: con gRPC, si InventoryService está caído, la confirmación falla igual.

Ejercicio 2 – ¿Qué ocurre si NotificationService está caído temporalmente?

Los mensajes **no se pierden**: quedan almacenados en la cola de RabbitMQ y se entregan cuando el servicio vuelve. OrdersService no se entera de la caída: sigue creando pedidos y respondiendo `201` sin demoras, porque solo depende del broker, no del consumidor.

> **Prueba realizada.** Con `notification-service` detenido se crearon 3 pedidos (los tres respondieron `201`). `rabbitmqctl list_queues` mostró `notifications.order-created 3 mensajes, 0 consumidores` y no se envió ningún correo. Al iniciar el servicio, la cola quedó en `0 mensajes, 1 consumidor` y llegaron los 3 correos a Mailpit, en orden.

Esto funciona porque se combinan varias decisiones:

- **Cola durable y mensajes persistentes** (`durable: true`, `persistent: true`): sobreviven incluso a un reinicio de RabbitMQ.
- **La cola la declaran ambos lados.** Si solo la declarara el consumidor y este nunca hubiera arrancado, el exchange no tendría una cola enlazada y RabbitMQ descartaría los mensajes en silencio.

- **Publisher confirms:** OrdersService considera publicado el mensaje solo cuando el broker confirma que lo guardó.
- **Ack manual después de enviar el correo:** si el servicio se cae a mitad del envío, el mensaje no confirmado vuelve a la cola y se reentrega.
- **Idempotencia:** como la reentrega puede producir duplicados, NotificationService recuerda los `messageId` ya procesados y no envía el mismo correo dos veces.
- **Dead-letter queue:** si un mensaje falla dos veces (por ejemplo, el servidor de correo rechaza la dirección), pasa a `notifications.order-created.dlq` en lugar de bloquear la cola.

Efectos a tener en cuenta. El cliente recibe el correo con retraso (consistencia eventual) y, si la caída es larga, la cola crece y consume memoria y disco en el broker; en producción se monitorea su tamaño y se pueden definir TTL o límites. Si el que cae es **RabbitMQ** (no el consumidor), la publicación falla: en el prototipo el error queda registrado en el log y el pedido se crea igual, por lo que ese aviso se pierde. La solución habitual es el **patrón Outbox**: guardar el evento en la misma transacción que el pedido y que un proceso aparte lo publique cuando el broker vuelva.

En comparación con el Ejercicio 1: si se hubiera notificado con una llamada REST síncrona, la caída de NotificationService habría hecho fallar o demorar la creación del pedido. La cola desacopla ambos servicios en el tiempo.

Ejercicio 3 – Diferencias entre usar Kafka y RabbitMQ en este escenario

En el Ejercicio 3 cada cambio de estado (`CREATED`, `CONFIRMED`, `SHIPPED`) se publica en el topic `orders.status-changed` y lo leen dos consumer groups: `notification-service` y `analytics-service`.

Aspecto	Con Kafka (implementado)	Si se hiciera con RabbitMQ
Agregar AnalyticsService	Solo se creó un nuevo consumer group. OrdersService no se modificó.	Habría que crear una cola nueva enlazada a un exchange fanout o topic, una por cada consumidor.
Historial	Los eventos quedan en el log después de leídos.	Cada cola borra el mensaje al confirmarlo; el historial no existe en el broker.
Consumidor nuevo o reconstrucción	Puede leer desde el offset 0 y procesar toda la historia.	Solo recibe lo publicado después de crear su cola; lo anterior se perdió para él.
Orden por pedido	Garantizado: la clave del mensaje es el id del pedido, así que todos sus eventos van a la misma partición.	Garantizado solo con un consumidor por cola; con varios en paralelo se puede procesar `SHIPPED` antes que `CONFIRMED`.
Consumidor caído	Al volver, sigue desde su último offset confirmado.	Los mensajes esperan en su cola (igual que en el Ejercicio 2).
Escalado de consumidores	Hasta un consumidor por partición dentro de un grupo (3 en el prototipo).	Cualquier cantidad de consumidores compitiendo en la misma cola.
Reintentos y errores	Sin DLQ nativa ni reintento por mensaje: hay que implementarlo (topics de retry).	Nack, reencolado y DLQ integrados.
Infraestructura	Más pesada (broker con KRaft, creación de topics, particiones).	Más liviana y simple de operar.

> **Prueba realizada.** AnalyticsService guarda sus métricas en memoria. Se reinició el contenedor (memoria vacía) y, como al iniciar vuelve a leer el topic desde el offset 0, en unos **5 segundos** reconstruyó exactamente las mismas métricas (9 eventos, 6 pedidos). Con RabbitMQ esto sería imposible: los mensajes ya consumidos no existen más. Además, al detener NotificationService, AnalyticsService siguió procesando eventos sin verse afectado, porque cada grupo tiene su propio offset.

Conclusión. Para los cambios de estado del pedido, Kafka es la mejor opción: son *hechos* que interesan a varios servicios, el orden por pedido importa y poder releer la historia permite sumar consumidores nuevos (analítica, auditoría) sin tocar al productor. RabbitMQ sigue siendo más adecuado para el Ejercicio 2, donde el mensaje es una *tarea puntual* (enviar un correo) que un solo trabajador debe ejecutar, con reintentos y DLQ incluidos. Por eso el prototipo usa ambos: el correo de "pedido recibido" sale por RabbitMQ, y NotificationService ignora el evento `CREATED` de Kafka para no enviarlo dos veces.

Decisiones de diseño del prototipo

- **Confirmación como paso separado** (`POST /orders/:id/confirm`): permite que el pedido exista en estado `CREATED`, que se notifique su recepción de forma asíncrona y que la verificación de stock síncrona ocurra justo antes de confirmar, como pide el enunciado.
- **Máquina de estados** `CREATED → CONFIRMED → SHIPPED`: cualquier otro salto se rechaza con 409, así nunca se publica un evento de una transición inválida.
- **Reserva atómica en InventoryService:** la consulta y el descuento del stock ocurren en una sola operación del servidor, lo que evita vender de más con pedidos concurrentes.
- **Consumidores idempotentes:** tanto NotificationService como AnalyticsService descartan mensajes con un id ya procesado, porque ambos brokers ofrecen entrega *al menos una vez*.
- **Apagado ordenado:** los consumidores de Kafka abandonan el grupo al recibir `SIGTERM`; sin esto, un reinicio tardaba unos 25 s más, hasta que el broker daba por muerto al miembro anterior y rebalanceaba.